LOOM V10

From blank bar to editable composition system

A practical tutorial for understanding the architecture, music data model, browser audio engine, analysis code, database, security, testing and optimization in the project.

1. The mental model

LOOM is easiest to understand as three systems joined together: a musical document model, a browser performance engine, and a measurement layer. The key design decision is that the song is not stored as a finished audio blob. It stays decomposable.

Layer	Responsibility	Primary data
Composition	What the user wrote	tracks, notes, timing, velocity
Performance	How the browser sounds it	Web Audio nodes, scheduling, synthesis
Analysis	What can be measured	density, pitch classes, velocity, novelty
Persistence	What survives a session	users, projects, revisions, notes, audits

Think of a project as a tree. A track owns an instrument and notes. A note is the smallest editable unit. Analysis reads those units; it does not replace them.

2. Environment and startup

The repository is intentionally dependency-light at runtime. SQLite lets the application run locally without provisioning a server database. PostgreSQL is the same relational model for deployment.

Commands

```
python -m venv .venv
pip install -r requirements.txt
python -m uvicorn backend.app:app --host 127.0.0.1 --port 8000
```

Why two database targets?

Local	Deployment
SQLite file, zero setup	PostgreSQL service, concurrent clients
Fast local iteration	Connection pooling and server durability
Easy tutorials/tests	Production-style relational database

3. Authentication: what happens on sign up

The browser first receives a CSRF token. The sign-up request must echo that token in a request header. The password is hashed with Argon2id; the raw password is never stored.

```
request -> CSRF check -> validate email/password -> hash password -> insert
user -> create random session token -> persist only token hash -> set
HttpOnly cookie
```

Session tokens are bearer credentials. Storing their hash instead of the raw token means a database leak does not immediately reveal active cookies.

Important boundary

Authentication is not authorization. Every project query also constrains owner_id, so knowing another project ID is not enough to read it.

4. Database schema and revision history

Table	Purpose	Important constraint
users	account identity	unique email
sessions	active login state	unique token hash + expiry
projects	current composition	owner_id + revision
project_versions	bounded restore points	project_id + revision
notes	user notes attached to a project	owner_id + project_id
audit_events	important changes	user_id + timestamp + event

Revision snapshots solve a practical problem: saving a project should not destroy the ability to recover a previous state. The final build keeps only the newest 40 snapshots so history cannot grow without bound.

5. The composition data model

```
{
 "tempo": 96,
 "tracks": [
   {"name":"Bass","instrument":"bass","notes":[
     {"pitch":48,"start":0.0,"duration":0.5,"velocity":0.72}
   ]}
 ]
}
```

A note has four essential values: pitch, start time, duration and velocity. The rest can be layered on later without breaking the base schema.

This makes editing cheap. Moving a note changes one number. Duplicating a pattern copies a small JSON structure. Exporting MIDI can reconstruct note-on and note-off events from the same representation.

6. Browser audio: from note to sound

The browser creates an AudioContext lazily after a user gesture. Each note becomes a scheduled oscillator or synthesized drum voice. Gain envelopes prevent clicks and keep peak amplitude under control.

```
frequency = 440 * 2 ** ((midi_pitch - 69) / 12)

at = audio_context.currentTime + start_seconds
oscillator.start(at)
oscillator.stop(at + duration)
```

Notice the separation: the server stores musical intent; the browser handles real-time audio. This avoids sending every note through a server round trip.

7. Piano roll interaction

The piano roll is a coordinate transform. Horizontal location maps to step/time. Vertical location maps to pitch. Clicking an empty cell creates a note; clicking an occupied cell removes it.

UI coordinate	Musical value
x	quantized start position
y	MIDI pitch
cell width	smallest timing subdivision
note block width	duration
block opacity/height	velocity or emphasis representation

A robust editor validates every created note before saving: pitch 0-127, non-negative start, positive duration and velocity 0-1.

8. Analysis algorithms

The analysis endpoint does not claim to infer taste or emotion. It calculates descriptive properties from the actual notes supplied.

Metric	Method	Typical complexity
Density	count notes by bar	O(n)
Pitch class	12-bin histogram	O(n)
Velocity profile	fixed histogram	O(n)
Novelty	compare ordered pitch-class changes	O(n log n) incl. ordering
2-D exploratory coordinates	standardized pitch/time coordinates	O(n)

The fixed-size pitch profile is intentionally simple and stable. More advanced models can be added later without changing the core project format.

9. Vectorization with NumPy

Instead of repeatedly looping through Python objects, the analysis converts numeric columns into NumPy arrays. This reduces Python-level overhead and makes the core operations operate on contiguous numeric memory.

```
starts = np.asarray([...], dtype=np.float64)
pitches = np.asarray([...], dtype=np.int16)
classes = np.mod(pitches, 12)
pitch_profile = np.bincount(classes, minlength=12)
```

This matters more as the note count grows. The algorithm still scales with the number of notes, but the constant factor is lower.

10. MIDI export

MIDI uses delta times rather than absolute time in the event stream. LOOM converts note positions into ticks, sorts note-on/note-off events, writes variable-length quantities, and wraps them in MThd/MTrk chunks.

Term	Meaning
PPQ	ticks per quarter note
VLQ	variable-length quantity for compact deltas
0x90	note-on status base
0x80	note-off status base
0xFF 0x2F 0x00	end-of-track event

11. Style laboratory

Style recipes are data, not prompts. A recipe contains tempo, meter, layers and a short structural description. That makes the teaching feature inspectable and editable.

Example: a house recipe can describe a four-to-the-floor kick, repeated harmonic cell, bass pocket and incremental timbral movement. The UI can then let you change one layer and immediately hear the consequence.

12. Provenance and real data procurement

The archive uses source-linked Mutopia metadata. The source manifest records the URL, use and refresh script. The refresh script can parse a saved fixture so the ingestion process can be tested even when network access is unavailable.

This is an important engineering rule: a real source should remain a real source. The application should not turn a metadata snapshot into a claim that it contains live audio or current facts.

13. Search and external systems

The core application does not require Elasticsearch, Kafka or a cloud API to function. Optional integration boundaries are documented separately because forcing every infrastructure component into the runtime would make the product harder to run and test.

System	Why it can be added
Elasticsearch	full-text and archive faceting at larger scale
Kafka	durable event fan-out for asynchronous workers
PostgreSQL	multi-user durable relational storage
Kubernetes	replicated/stateless deployment
Terraform	repeatable infrastructure definition
BigQuery	analytical warehouse export

14. Security checklist

- Argon2id password hashing.
- Random server sessions; only token hashes persist.
- CSRF token required on state-changing routes.
- HttpOnly + SameSite cookies.
- Bounded request arrays and string lengths.
- Owner predicate on every project operation.
- Rate-limited login/signup attempts.
- Revision history is bounded.
- No arbitrary file execution endpoint.
- External procurement is outside the critical request path.

15. Testing strategy

Tests should fail for the same reasons users would experience failures. LOOM therefore tests API routing, authentication boundaries, project ownership, analysis edge cases and data manifests. The verification script also runs Python compilation and all pytest tests.

```
python -m compileall -q backend
python -m pytest -q
python scripts/verify.py --json
```

16. Performance and storage reasoning

Problem	Technique
Repeated analysis of small numeric arrays	NumPy vectorization
Large project history	keep latest 40 snapshots
Server-side audio latency	Web Audio playback in browser
Optional data fetches slowing requests	offline snapshot + separate refresh script
Growing authentication attempts	bounded in-process limiter

Do not optimize by making the code obscure. The project prefers measurable simple paths over clever abstractions that are difficult to maintain.

17. Debugging walkthrough

When the piano roll looks wrong, check the data before changing CSS. First inspect pitch, start and duration. Then inspect the mapping from cell index to musical time. Finally inspect the render loop. The same principle applies to the analysis charts: verify the returned JSON before debugging canvas drawing.

For database problems, reproduce with a fresh SQLite file, inspect the request payload, then run the same test against PostgreSQL. Never “fix” an authorization bug by hiding a button; fix the server predicate.

18. Portfolio explanation

A strong technical explanation is: “LOOM is an editable music system whose source of truth is structured musical data. The browser performs low-latency synthesis; Python performs deterministic analysis; PostgreSQL persists users and revisions; security and provenance are explicit.”

That sentence demonstrates software architecture, music technology, data modelling and engineering judgement in one breath.

19. Advanced roadmap

• Rust/WASM implementation of selected DSP kernels after profiling proves a Python/browser bottleneck.

• MIDI import with validation and deduplication.

• Polyphonic sampler with bounded memory cache.

• Track-level automation curves.

• Audio waveform upload pipeline with sandboxing.

• Elastic index for archive search at larger corpus size.

• Kafka outbox publisher for asynchronous analytics.

• Property-based testing for MIDI serialization.

20. Learning roadmap

Stage	Learn	Build
Beginner	JSON, HTTP, SQL, DOM	edit notes and save a project
Intermediate	Web Audio, NumPy, SQLAlchemy	play and analyze a pattern
Advanced	sessions, indexing, complexity	private revisioned projects
Portfolio	deployment, CI/CD, security	ship the full release
Research	DSP, embeddings, sequence models	add a measured new analysis module

21. Final checklist

- Can a new user understand the product without reading the code?
- Can a user create, modify, save and restore a project?
- Can the browser play the project without a hosted AI service?
- Can the analysis explain what it measured?
- Can another user access someone else's project? It should not.
- Can the data source be traced?
- Can the database be switched to PostgreSQL?
- Can the test suite be run from a clean environment?
- Can the repository explain its deliberate non-choices?

The purpose of the notes is not to memorize the implementation. It is to give you a map of why each part exists, what problem it solves, and where you can extend it next.